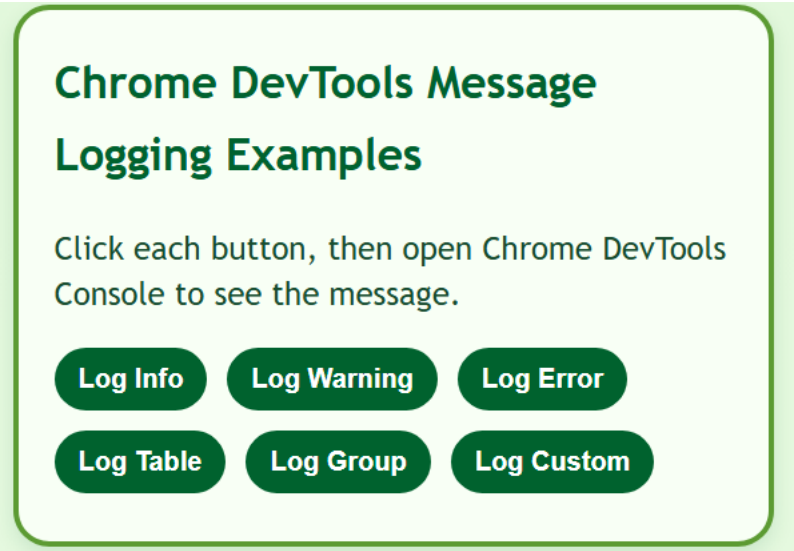
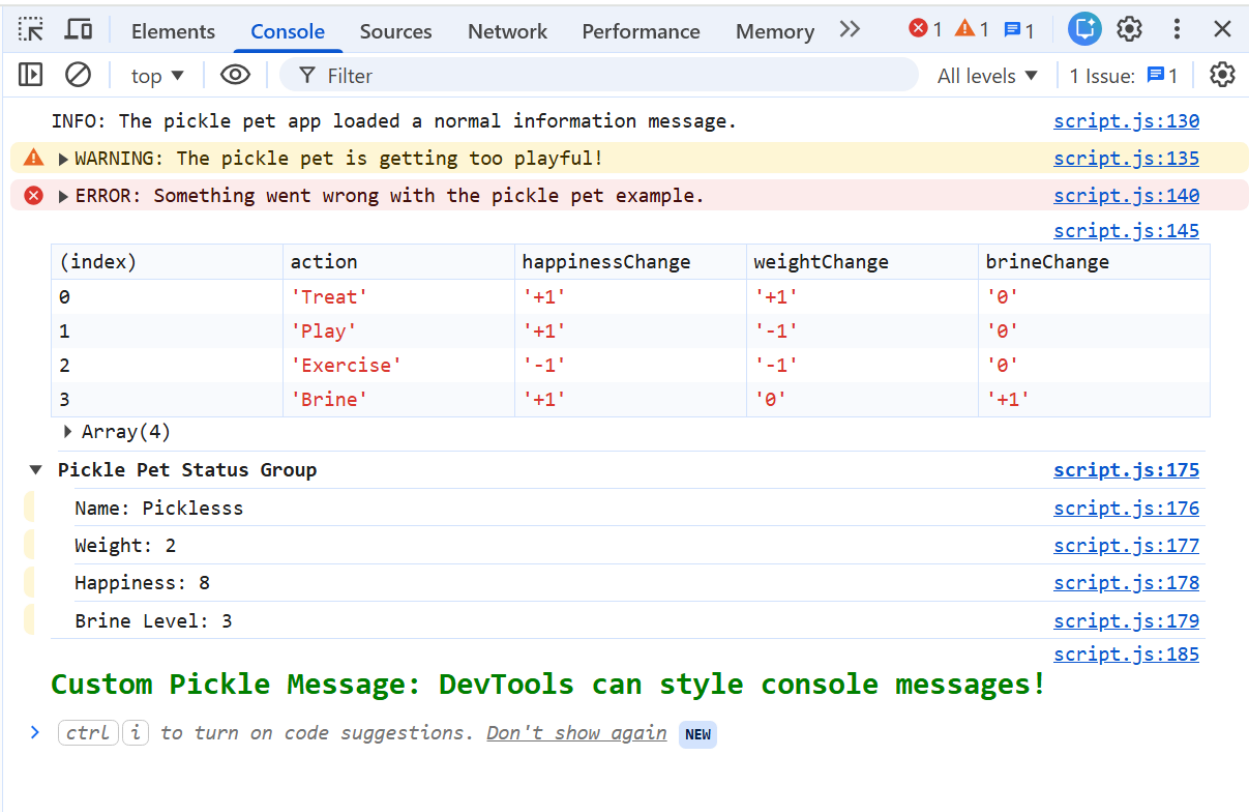
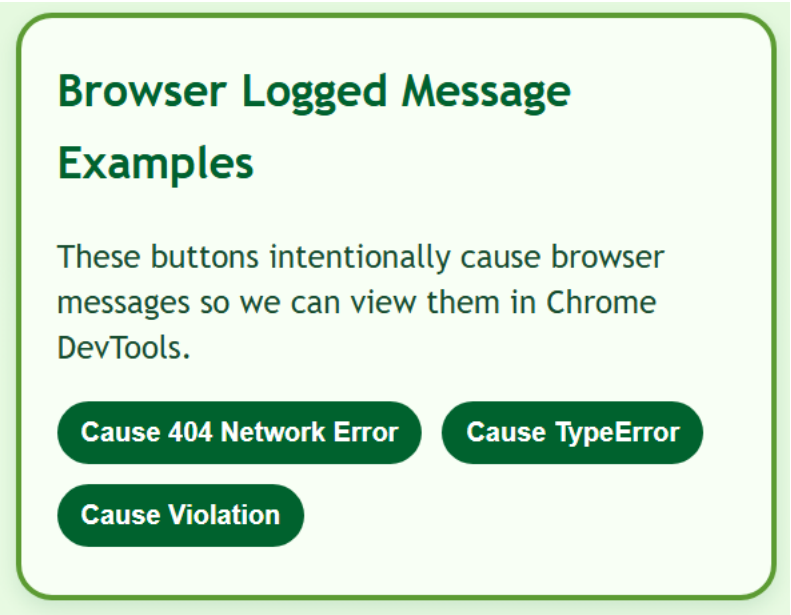
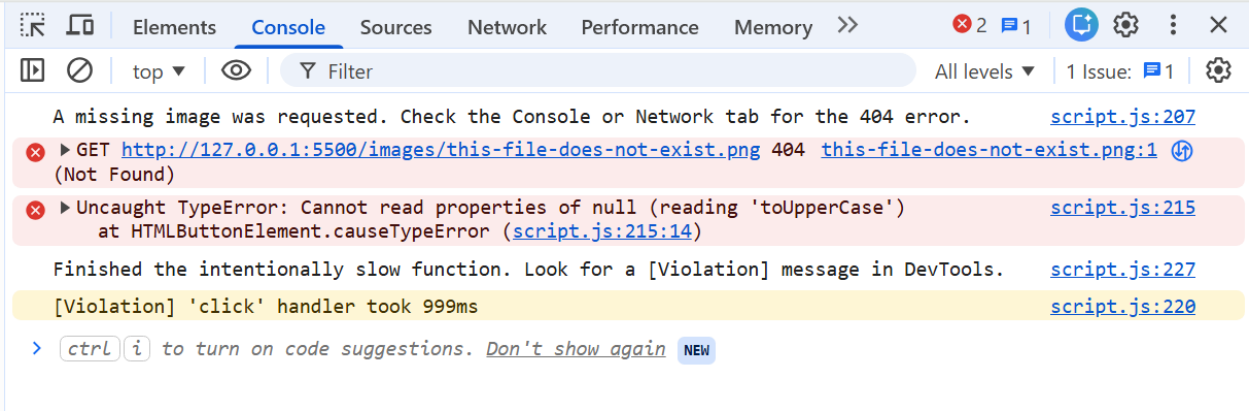
I implemented a Message Logging section in my Project 2 page by adding a new DevTools practice area with six buttons: Log Info, Log Warning, Log Error, Log Table, Log Group, and Log Custom. Each button is connected to a JavaScript function using addEventListener(). When the buttons are clicked, they run different Console methods such as console.info(), console.warn(), console.error(), console.table(), console.group(), and a styled console.log() message. This implementation demonstrates how different types of messages appear in the Chrome DevTools Console.



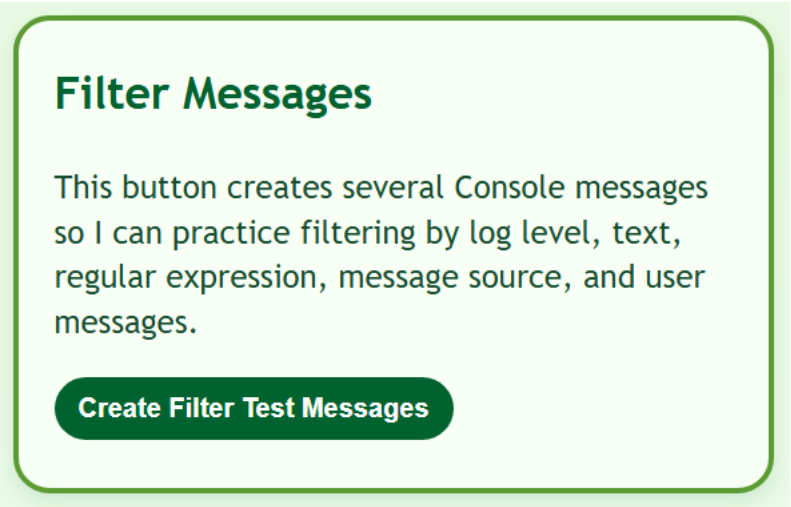


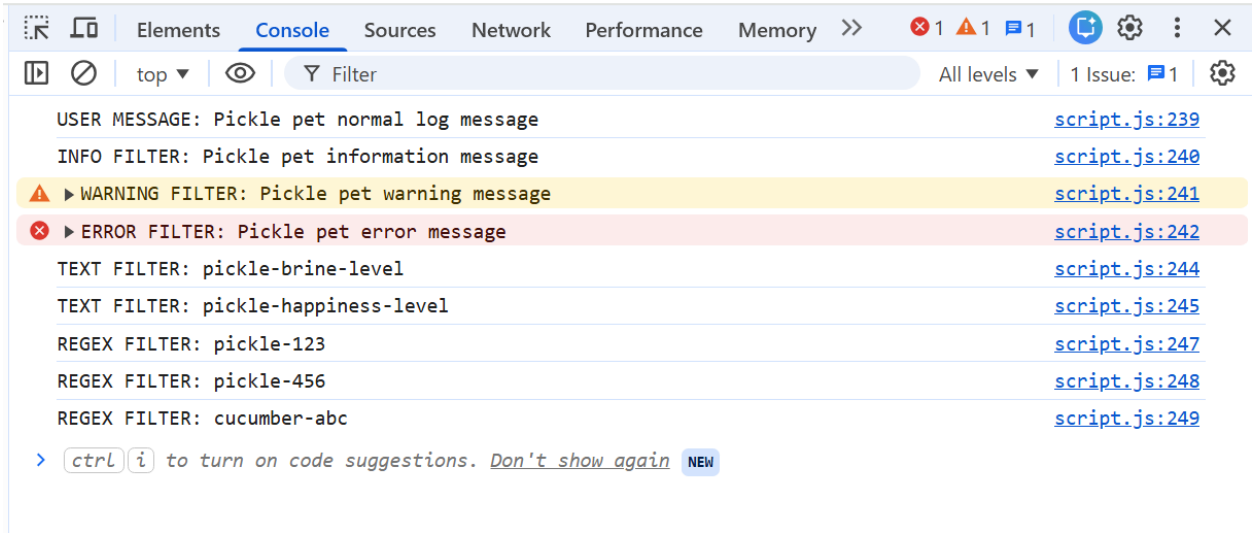
I implemented a Browser Logged Message Examples section to show messages created by the browser instead of only messages created by my own console.log() code. I added three buttons: Cause 404 Network Error, Cause TypeError, and Cause Violation. The 404 button tries to load an image file that does not exist, causing Chrome to log a failed network request. The TypeError button runs code that calls a method on a null value, causing a JavaScript error. The Violation button runs a slow loop that blocks the page briefly, allowing Chrome to show a performance warning. This section demonstrates browser-generated messages in DevTools.



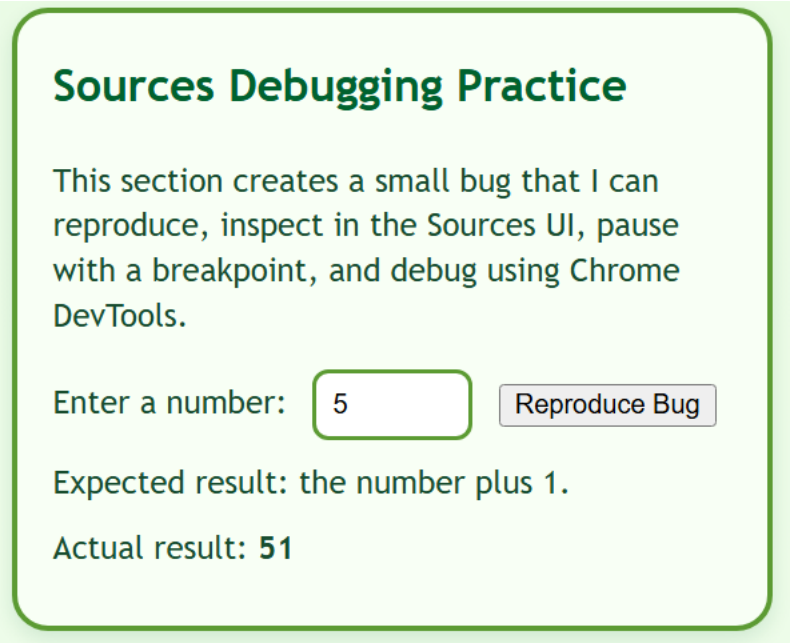


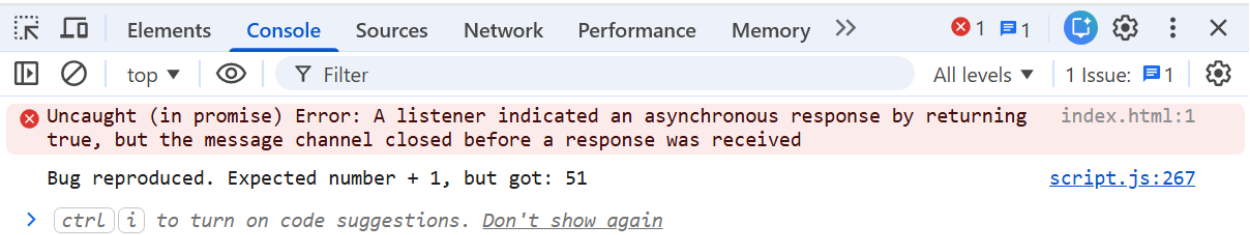
I implemented a Filter Messages section by adding a button called Create Filter Test Messages. When clicked, it prints several Console messages with clear labels for testing filters. The messages include normal logs, info messages, warnings, errors, text based labels, and regular expression examples. This gave me sample Console output that I could filter in Chrome DevTools by log level, text, regular expression, message source, and user messages. This implementation shows how Console filters help separate important debugging messages from a large amount of output.



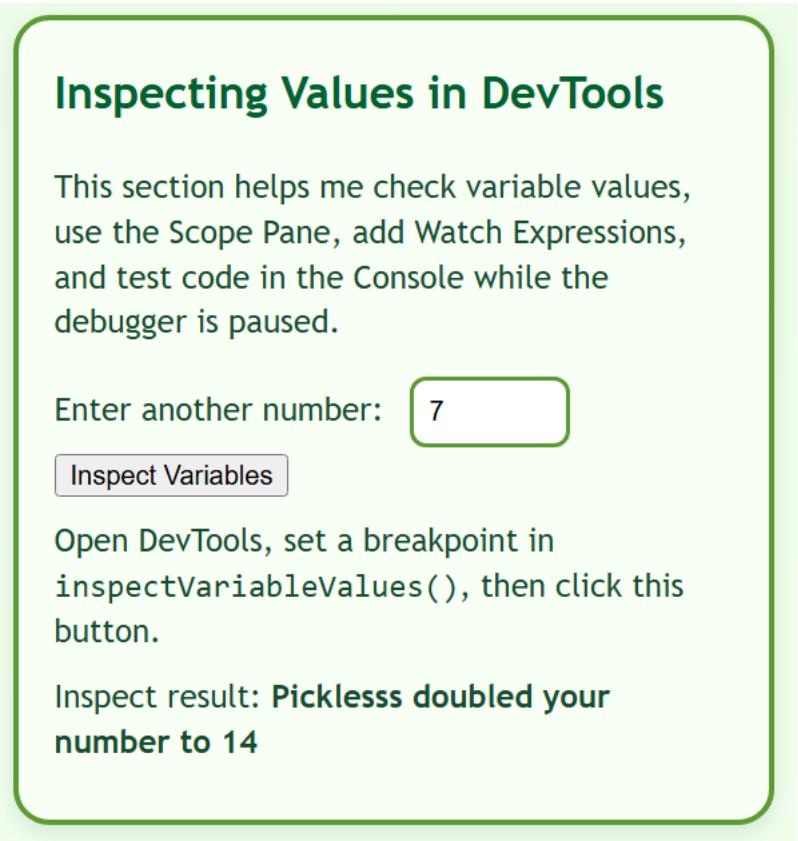


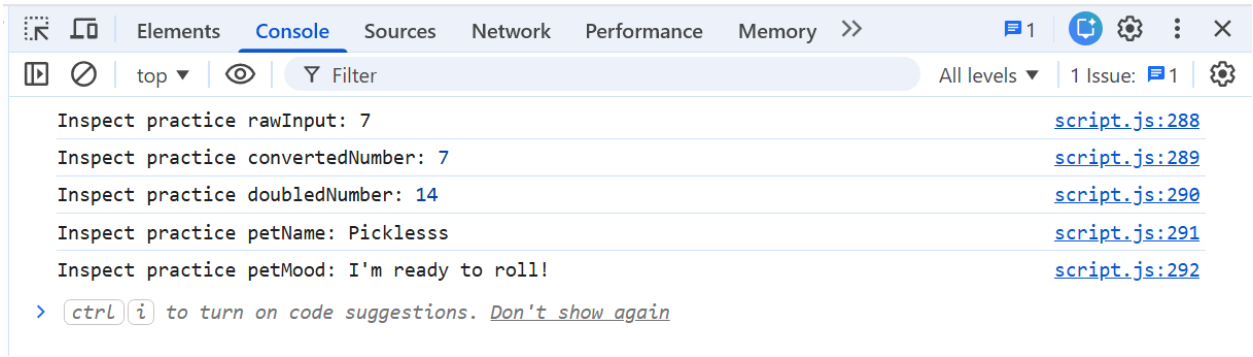
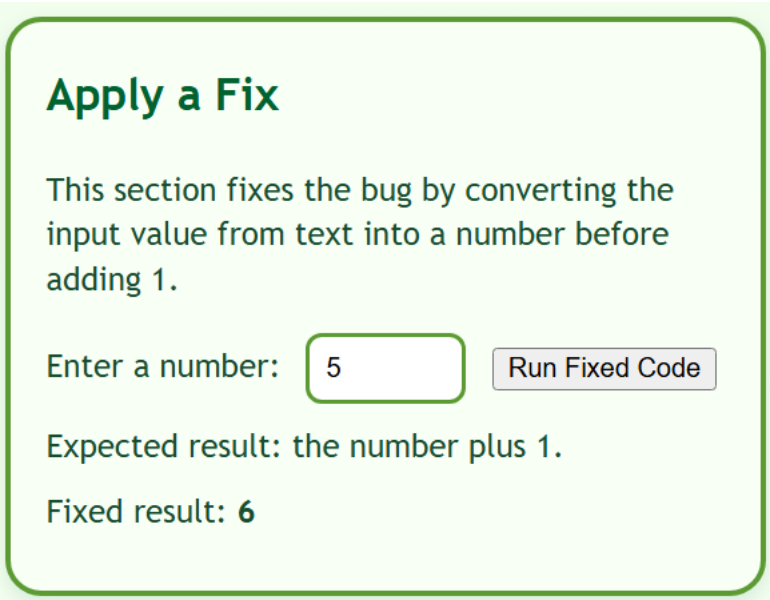
I implemented a Sources Debugging Practice section by adding a small form with an input box, a Reproduce Bug button, and an output area. The JavaScript function intentionally contains a bug where the input value is read as a string. Because of this, entering 5 and adding 1 displays 51 instead of 6. This bug was created on purpose so I could open the Sources panel in Chrome DevTools, locate the JavaScript file, set a line-of-code breakpoint, pause the code, and inspect the problem while the function was running.





I implemented an Inspecting Values in DevTools section by adding another input box, an Inspect Variables button, and an output area. The connected JavaScript function creates multiple variables, including the raw input value, converted number, doubled number, pet name, and pet mood. This setup allowed me to pause the code in the Sources panel and inspect those variables using the Scope Pane. I also used Watch Expressions to track selected values and tested expressions directly in the Console while the debugger was paused. This implementation demonstrates how DevTools can be used to understand what values exist at a specific point in the program.





I implemented an Apply a Fix section to correct the bug from the Sources Debugging Practice section. I added a new input box, a Run Fixed Code button, and a fixed result display. In the corrected JavaScript function, the input value is converted from a string into a number using Number(userInput) before adding 1. This prevents string concatenation and makes the calculation work correctly. After the fix, entering 5 displays 6 instead of 51. This implementation shows how DevTools can be used to reproduce a bug, inspect the cause, and verify that the corrected code works.